

PHASE 3: Governance & Retrieval Optimization

***Goal:** Securely store processed data in Delta Lake, optimize the retrieval engine using re-ranking and filtering, and evaluate application performance using MLflow metrics.*

1. Storage and Governance (Delta Lake & Unity Catalog)

Core Concept

Once text is parsed and chunked, it must be stored in a scalable, governed repository before it can be embedded and searched. Unity Catalog governs access to Delta Lake tables, allowing you to manage Access Control Lists (ACLs) so that the RAG model only retrieves data the end-user is authorized to see.

How it Works on Databricks

On Databricks, chunked data is written to a Delta table governed by Unity Catalog. You then create a **Databricks AI Search** (formerly Vector Search) index directly from this Delta table. The index automatically syncs with the underlying Delta table (either on a triggered or continuous basis), computing embeddings using a specified foundation model and keeping the vector database up-to-date.

When to Use vs. Alternatives

Use Databricks AI Search synced with a Delta table when building production RAG apps within the Databricks ecosystem, as it removes the need to manually manage an external vector database (like Pinecone) and keeps your data governed under one unified Unity Catalog roof. Use external vector databases only if your organization has pre-existing architectural mandates outside of Databricks.

Common Mistakes

A major mistake is forgetting to include metadata columns (like `document_type`, `date`, or `author`) in the source Delta table. Metadata filtering is considered the "biggest lever for retrieval quality," dramatically improving precision by eliminating irrelevant results before the vector search even runs.

```

from databricks.vector_search.client import VectorSearchClient

vsc = VectorSearchClient()

# Create an AI Search index synced from a Delta table in Unity Catalog
vsc.create_delta_sync_index(
    endpoint_name="my_vector_endpoint",
    source_table_name="main_catalog.rag_schema.chunked_docs",
    index_name="main_catalog.rag_schema.chunked_docs_index",
    pipeline_type="TRIGGERED",
    primary_key="chunk_id",
    embedding_source_column="chunked_text",
    embedding_model_endpoint_name="databricks-bge-large-en"
)

```

Function Explanation: `create_delta_sync_index` automatically provisions a vector search index linked to your Delta table (`source_table_name`). It tells Databricks to take the text in `embedding_source_column`, pass it to the `embedding_model_endpoint_name` to generate vectors, and sync updates automatically based on the `pipeline_type`.

2. Retrieval Optimization (Re-ranking)

Core Concept

Vector databases retrieve chunks based on mathematical similarity (e.g., L2 distance or Cosine similarity). However, similarity does not always equal relevance. **Re-ranking** introduces a secondary processing step where a smaller, more powerful cross-encoder model evaluates the initially retrieved chunks and re-orders them based on their exact contextual relevance to the user's specific query.

How it Works on Databricks

Databricks AI Search supports native re-ranking. First, you run a keyword or hybrid semantic search to retrieve a broad set of results (e.g., top 20 chunks). Then, a cross-encoder model (like `mxbai-rerank`) scores the exact semantic relationship between the query and each chunk, outputting the most highly relevant top K chunks (e.g., top 5) to feed to the LLM.

When to Use vs. Alternatives

Use Re-ranking when dealing with complex, domain-specific queries where keyword or standard embedding searches yield contextually similar but factually unhelpful results. An alternative is pure **Hybrid Search** (combining vector similarity and BM25 keyword matching). Databricks recommends using *both* together: retrieve with Hybrid Search, then optimize the order with Re-ranking.

3. Audit & Evaluation

Core Concept

You cannot improve what you cannot measure. Databricks advocates for a deterministic, metrics-driven approach using an **Evaluation Set** (a curated set of queries and ground-truth answers). The performance is audited using LLM Judges that evaluate standard RAG metrics like Precision, Recall, and Groundedness.

How it Works on Databricks

Databricks provides **Agent Evaluation** (integrated with MLflow). It provides built-in LLM judges (using models like GPT-4) to grade your pipeline's output. Key metrics include:

- **Retrieval Precision:** What percentage of retrieved chunks are relevant?
- **Document Recall:** What percentage of ground-truth documents were successfully retrieved?
- **Retrieval Groundedness:** Does the generated response rely entirely on the retrieved context (no hallucinations)?

Common Mistakes

A common mistake is testing on the same data used to refine the model (overfitting). Databricks strictly recommends splitting your evaluation set into Training (~70%), Test (~20%), and Validation (~10%) sets.

```
import mlflow

# Evaluate RAG application performance using MLflow built-in judges
results = mlflow.evaluate(
    data=eval_set_df,
    model_type="question-answering",
    extra_metrics=[
        mlflow.metrics.genai.answer_correctness(),
        mlflow.metrics.genai.answer_relevance()
    ]
)

print(results.metrics)
```

Function Explanation: `mlflow.evaluate()` automates the auditing process. It takes your application's evaluation dataset (`eval_set_df` containing queries, model outputs, and retrieved context) and applies built-in GenAI metrics like `answer_correctness` to statistically prove the quality of your RAG pipeline before deploying it to production.

Sources (Official Databricks Documentation)

- AI Search: <https://docs.databricks.com/aws/en/ai-search/ai-search>
- Retrieval Quality Guide: <https://docs.databricks.com/aws/en/ai-search/retrieval-quality>
- Agent Evaluation Metrics: <https://docs.databricks.com/aws/en/agents/tutorials/ai-cookbook/evaluate-assess-performance>
- Scorers and LLM Judges: <https://docs.databricks.com/aws/en/mlflow3/genai/eval-monitor/concepts/scorers>